Boxed-Error Reweighting for On-Policy Self-Distillation

Kong Jing[1*], Sun YuXin[1*], Shen Lixin[1*], Chen Yitong[1*]

[1]College of Computing and Data Science, Nanyang Technological University

Kong Jing (G2600255B, JING033@e.ntu.edu.sg); Sun YuXin (G2509636J, Suny0094@e.ntu.edu.sg);
Shen Lixin (G2602590K, SHEN0281@e.ntu.edu.sg); Chen Yitong (G2606885D, chen2216@e.ntu.edu.sg)

Abstract

On-policy self-distillation (OPSD) trains one language model as both student and teacher. The student samples a solution from the problem alone; the teacher, which also sees the reference solution, supplies token-level supervision on that same trajectory. The original loss never checks whether the sampled answer is correct. We multiply that loss by a sequence weight from a boxed-answer verifier: a completed wrong answer is upweighted, while truncated traces and already-correct traces stay at weight one. On Qwen3-1.7B with a 100-step LoRA budget, the change lifts HMMT25 Avg@12 from 22.8 to 27.8 relative to a reproduced OPSD run. AIME24 and AIME25 stay at or below that run. An ablation that also upweights truncations is reported alongside the default boxed-error rule.

1 Introduction

Post-training for mathematical reasoning is usually some mix of supervised imitation, reinforcement learning from verifiable rewards (RLVR), and distillation from a stronger teacher (Hinton, Vinyals, and Dean 2015; Ouyang et al. 2022; Shao et al. 2024; Agarwal et al. 2024). Supervised fine-tuning is off-policy. Group-relative policy optimization (GRPO) and related RLVR methods give a sequence-level scalar, and the gradient vanishes when a group is uniformly correct or uniformly wrong (Shao et al. 2024; DeepSeek-AI 2025). Distillation is denser, but the usual recipes need a separate teacher and often train on teacher-generated text.

OPSD (Zhao et al. 2026) drops the external teacher. One network plays both roles through different prompts: the student sees the problem, and the teacher also sees a reference solution while scoring the student's own tokens. Training uses one rollout per problem and a few hundred optimizer steps. The authors report large gains on AIME and HMMT with Qwen3 models.

Two observations motivate the present work. First, OPSD's loss does not look at whether the student answer is right. The original paper lists this as a limitation: the framework "does not explicitly leverage correctness verification of generated answers." Second, a single-GPU reproduction of the Qwen3-1.7B recipe recovers the paper's AIME24/25 step-100 numbers to about one point, but misses the reported HMMT25 gain ($23.1 \to 29.2$). Local OPSD is essentially flat on HMMT ($23.06 \to 22.78$). If a correctness signal is going to matter, this is the suite where it should show up.

After each student rollout we compute a binary reward R with the same boxed-answer extractor and math-verify checker used at evaluation, then scale the original per-token KL by a sequence weight $w(R)$. The teacher, LoRA layout, clip, and sampling protocol stay as in OPSD. Upweighting every incorrect rollout also upweights truncated completions that never boxed an answer; with a 1024-token cap and student thinking turned off, those traces are common. The default rule upweights only completed but incorrect rollouts. We call this boxed-error reweighting (BER-OPSD). Concurrent work by Wang et al. uses the name Outcome-Guided On-Policy Self-Distillation and the abbreviation OG-OPSD for a different method that switches the divergence and the distilled prefix using outcome rewards and teacher entropy (Wang et al. 2026). The KL term here is unchanged; only the sequence weight depends on R.

On the same $1\times$A800, 100-step LoRA budget used for the OPSD reproduction, boxed-error reweighting raises HMMT25 Avg@12 by 5.0 points over local OPSD and 4.7 points over the base model. AIME24 and AIME25 fall slightly. The rest of the paper reports all three suites and both weighting rules.

2 Background

On-policy distillation. Given a teacher p_T and student p_S, sequence-level distillation matches next-token distributions (Kim and Rush 2016; Gu et al. 2024). On-policy variants sample $y \sim p_S(\cdot \mid x)$ and apply the divergence on those tokens, reducing exposure bias (Agarwal et al. 2024). OPSD instantiates both distributions from one model p_θ by changing only the prompt: $p_S(\cdot \mid x) = p_\theta(\cdot \mid x)$ and $p_T(\cdot \mid x, y^\star) = p_\theta(\cdot \mid x, y^\star)$, with $y^\star$ a reference solution. The student generates $\hat{y} \sim p_S$; both heads then evaluate $\hat{y}$, and training minimizes a token-averaged forward KL along $\hat{y}$ (the released code uses β=0 in a generalized JSD, i.e. $\mathrm{KL}(p_T \| p_S)$), with a per-

*These authors contributed equally.

token clip that stops stylistic tokens from dominating (Zhao et al. 2026).

Verifiable rewards. RLVR assigns $R \in \{0, 1\}$ by checking a boxed answer against gold (Cobbe et al. 2021; Shao et al. 2024). GRPO estimates advantages within a group of rollouts; the gradient vanishes when all rewards in the group are equal. OPSD is dense and uses one sample, with no outcome check. BER-OPSD keeps that single rollout and the same KL, and uses R only as a sequence weight.

3 Boxed-Error Reweighting

Let $D_{\mathrm{KL}}^{(n)}$ be the clipped token KL at position n of a student completion $\hat{y}$. Standard OPSD uses

$$\ell(\hat{y}) = \frac{1}{|\hat{y}|} \sum_{n} D_{\mathrm{KL}}\big(p_T(\cdot \mid x, y^\star, \hat{y}_{<n}) \,\|\, p_S(\cdot \mid x, \hat{y}_{<n})\big). \tag{1}$$

We leave this term unchanged and multiply it by a sequence weight. After decoding $\hat{y}$ we extract the last \boxed{·} span (searching after </think> when present) and set $R = \mathbf{1}[\mathrm{verify}(\hat{y}, y^\star)]$, using math-verify with a normalized-string fallback. Let $B = \mathbf{1}[\hat{y}$ contains a boxed span]. Two gates are compared:

$$w_{\mathrm{all}} = 1 + \lambda(1 - R), \tag{2}$$

$$w_{\mathrm{box}} = 1 + \lambda\, B\, (1 - R). \tag{3}$$

The training loss is $\mathbb{E}[w\, \ell(\hat{y})]$. When $w \equiv 1$ the estimator is identical to OPSD, including the token-micro average: sequence weights broadcast over tokens, so all-ones recovers the original reduction.

Equation (2) (All-Error) doubles every incorrect sequence, including truncated text with B=0. In our 100-step runs the boxed rate is about 0.48 and the true-correct rate about 0.37, so roughly half of the extra mass lands on incomplete prefixes. Equation (3) (Boxed-Error, the default) applies the extra factor only to completed-but-wrong attempts. Truncations keep weight 1; already-correct rollouts also keep weight 1, so the teacher can still reshape style on successes. We use λ=1 unless noted. Algorithm 1 is the training step; lines after the OPSD rollout are the only addition.

There is no group, no advantage normalization, and no policy-gradient term on R. Gradients still flow through the student logits of the KL. R is treated as a constant with respect to θ, since the discrete verifier is non-differentiable. The implementation adds one flag to the public OPSD trainer; λ=0 recovers the original path.

Algorithm 1 BER-OPSD training step

Input: batch $\{(x_i, y_i^\star)\}$, p_θ, λ
Output: reweighted KL loss

```
for all (x, y*) in the minibatch do
    Sample ŷ ~ p_θ(· | x) {student; no gold}
    Teacher logits: p_θ(· | x, y*) with LoRA disabled
    Student logits: p_θ(· | x) with LoRA enabled
    ℓ ← clipped token KL(p_T ‖ p_S) on ŷ
    R ← verify(ŷ, y*);  B ← 1[boxed]
    w ← 1 + λ(1 − R) or 1 + λB(1 − R)
    accumulate w · ℓ
end for
update LoRA parameters of p_θ
```

4 Experiments

4.1 Protocol

All methods use Qwen3-1.7B (Yang et al. 2025) with LoRA rank 64, α=128, dropout 0.05, on the OpenThoughts math split of 29,434 problems (Guha et al. 2025). We follow the OPSD 1.7B recipe: student thinking off, teacher thinking on, fixed teacher (base weights, adapters disabled), forward KL, token clip 0.05, temperature 1.1 at train time, max completion 1024, max length 20,000, learning rate $5{\times}10^{-6}$, per-device batch 2, gradient accumulation 2 (effective batch 4), 100 steps, vLLM colocated rollouts (Kwon et al. 2023; Hu et al. 2022). Training runs on one A800 80GB GPU in about 20 minutes. The original OPSD numbers were trained on 4×H100 with effective batch 32. The comparisons below are against a reproduction trained and evaluated under the same 1-GPU budget.

Evaluation matches the OPSD README: thinking on, temperature 1.0, top-p=0.95, top-k off, 38,912 new tokens, 12 samples per problem, Avg@12 as the primary metric, plus Pass@12 and majority-vote@12. Suites are AIME24, AIME25, and HMMT February 2025, 30 problems each (Mathematical Association of America 2025; Harvard–MIT Mathematics Tournament 2025). Checkpoints are merged into the base before serving. Following the OPSD README we use a single decoding seed; Avg@12 typically moves by a point or two across runs.

Baselines. Base is the unmodified Qwen3-1.7B instruct model. OPSD is our 100-step reproduction (repro_100_gpu0). A 100-step GRPO run with 1024-token completions would not match the official GRPO recipe (eight 16k-token rollouts per prompt), so we leave GRPO out. SFT on the same traces shortens test-time reasoning in the OPSD paper; we did not rerun it.

4.2 Main Results

Table 1 reports Avg@12. Local OPSD already matches the published 1.7B step-100 AIME numbers to about one point (AIME24 56.1 vs. 57.2; AIME25 42.2 vs. 41.1) and matches the published base on HMMT25 (23.06 vs. 23.1). The published HMMT25 step-100 score of 29.2 is not recovered.

Boxed-Error is the default method. Relative to local OPSD it moves HMMT25 from 22.8 to 27.8 Avg@12 (82/360 correct solutions to 100/360), Pass@12 from 43.3 to 56.7, and majority vote from 26.7 to 33.3 (Table 2). The paper's 4-GPU OPSD score on this suite is 29.2. AIME24 drops 0.8 and AIME25 drops 3.9 Avg@12.

Method	AIME24	AIME25	HMMT25	Macro	Δ HMMT	vs. Base
Base Qwen3-1.7B	50.3	33.9	23.1	35.7	—	—
OPSD (local, 100 step)	56.1	42.2	22.8	40.4	−0.3	+4.6
BER-OPSD All-Error	57.5	40.0	23.6	40.4	+0.5	+4.6
BER-OPSD Boxed-Error	55.3	38.3	27.8	40.5	**+4.7**	**+4.7**
Paper OPSD @100 on 4×H100: 57.2 / 41.1 / 29.2.						

Table 1: Avg@12 (%) under the paper evaluation protocol. Local OPSD and both BER-OPSD variants share the 1×A800, 100-step LoRA budget. Macro is the unweighted mean of the three suites. "vs. Base" is the macro gap to Qwen3-1.7B. Best local number in each column is bold. Original 4-GPU OPSD scores are listed below the table for reference.

	AIME24	AIME25	HMMT25
OPSD Pass@12	76.7	66.7	43.3
Boxed-Error Pass@12	76.7	70.0	56.7
OPSD Maj@12	70.0	60.0	26.7
Boxed-Error Maj@12	70.0	50.0	33.3

Table 2: Pass@12 and majority-vote@12 (%). Boxed-Error matches OPSD on AIME24, raises AIME25 Pass@12 while Maj@12 falls, and raises both metrics on HMMT25.

The three-suite mean is 40.5 versus OPSD's 40.4; over 1080 graded solutions the counts are 437 versus 436.

All-Error upweights truncated text as well. It is the only variant that beats local OPSD on AIME24 (57.5 vs. 56.1, Pass@12 80.0 vs. 76.7). It loses 2.2 on AIME25, gains 0.8 on HMMT, and ties OPSD on the macro mean and on the 1080-solution count (436–436). Extra loss mass therefore lands on prefixes that never boxed an answer, so we keep Boxed-Error as the default.

OPSD and both reweighted runs stay well above the base model on AIME (roughly +5 to +8 Avg@12).

4.3 Training Signal

On-policy completions in this recipe are short and often unfinished. Across the All-Error run the mean boxed rate is 0.48 and the mean verified-correct rate is 0.37. The Boxed-Error run logs a boxed-error rate (completed-but-wrong) around 0.10–0.13 per logging window, so the extra weight is sparse. Firing on about one in eight traces concentrates the extra loss on completed mistakes rather than on the large truncated mass.

The wall-clock cost of the verifier is negligible next to vLLM generation. End-to-end training time stays at about 20 minutes, the same as OPSD.

5 Analysis and Limitations

Why HMMT moves and AIME does not. AIME24/25 were already the better numbers for local OPSD, so a sparse extra weight has less room there. HMMT25 is harder for this 1.7B checkpoint (base 23.1), and local OPSD did not help it. Reweighting completed mistakes spends the same 100 steps on traces that reached an answer and got it wrong, which is where a privileged teacher can actually correct the student. This is consistent with the scores and with the training statistics; we do not isolate a token-level mechanism.

Single-seed evaluation. The OPSD README notes that Avg@12 from one seed will move by a point or two. The 0.8 AIME24 drop is in that range. The 5.0 HMMT lift also appears in Pass@12 and majority vote. We only report one seed.

Limitations. Results are for Qwen3-1.7B, LoRA, and 100 steps. The verifier only sees a boxed span, so a wrong but well-reasoned trace with no box is treated like a truncation. The discrete R is not token-level credit assignment. We did not add GRPO-style group advantages, a process reward model, or a different evaluation protocol. Scaling the same weight to 8B, to thinking-on student rollouts, or to a full GRPO token budget is left open.

6 Conclusion

OPSD already gives a dense, on-policy, single-model training signal, without using whether the student was right. Boxed-error reweighting multiplies that signal by a verifier weight and leaves truncated traces at weight one. Under a matched local budget, HMMT25 Avg@12 rises by five points; AIME24 and AIME25 do not. The change is a few dozen lines on the public trainer, and λ=0 restores the original algorithm.

References

Agarwal, R.; Vieillard, N.; Zhou, Y.; Stanczyk, P.; Ramos, S.; Geist, M.; and Bachem, O. 2024. On-Policy Distillation of Language Models: Learning from Self-Generated Mistakes. In International Conference on Learning Representations.

Cobbe, K.; Kosaraju, V.; Bavarian, M.; Chen, M.; Jun, H.; Kaiser, L.; Plappert, M.; Tworek, J.; Hilton, J.; Nakano, R.; et al. 2021. Training Verifiers to Solve Math Word Problems. In arXiv preprint arXiv:2110.14168.

DeepSeek-AI. 2025. DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning. arXiv:2501.12948.

Gu, Y.; Dong, L.; Wei, F.; and Huang, M. 2024. MiniLLM: Knowledge Distillation of Large Language Models. arXiv:2306.08543.

Guha, E.; et al. 2025. OpenThoughts: Data Recipes for Reasoning Models. arXiv:2506.04178.

Harvard–MIT Mathematics Tournament. 2025. HMMT February 2025 Contest. https://www.hmmt.org/. Accessed: 2026-09-16.

Hinton, G.; Vinyals, O.; and Dean, J. 2015. Distilling the Knowledge in a Neural Network. In NIPS Deep Learning and Representation Learning Workshop.

Hu, E. J.; Shen, Y.; Wallis, P.; Allen-Zhu, Z.; Li, Y.; Wang, S.; Wang, L.; and Chen, W. 2022. LoRA: Low-Rank Adaptation of Large Language Models. In International Conference on Learning Representations.

Kim, Y.; and Rush, A. M. 2016. Sequence-Level Knowledge Distillation. In Proceedings of the 2016 Conference on Empirical Methods in Natural Language Processing, 1317–1327.

Kwon, W.; Li, Z.; Zhuang, S.; Sheng, Y.; Zheng, L.; Yu, C. H.; Gonzalez, J. E.; Zhang, H.; and Stoica, I. 2023. Efficient Memory Management for Large Language Model Serving with PagedAttention. In Proceedings of the 29th Symposium on Operating Systems Principles.

Mathematical Association of America. 2025. AIME 2024 and 2025 Contest Problems. https://artofproblemsolving.com/wiki/index.php/AIME_Problems_and_Solutions. Accessed: 2026-09-16.

Ouyang, L.; Wu, J.; Jiang, X.; Almeida, D.; Wainwright, C. L.; Mishkin, P.; Zhang, C.; Agarwal, S.; Slama, K.; Ray, A.; et al. 2022. Training Language Models to Follow Instructions with Human Feedback. In Advances in Neural Information Processing Systems.

Shao, Z.; Wang, P.; Zhu, Q.; Xu, R.; Song, J.; Bi, X.; Zhang, H.; Zhang, M.; Li, Y. K.; Wu, Y.; and Guo, D. 2024. DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models. arXiv:2402.03300.

Wang, Z.; Luo, M.-L.; Hong, Y.; Zhou, Z.-H.; Ye, B.; Zhao, J.; Tong, X.; Zhang, M.-L.; and Wei, T. 2026. Outcome-Guided On-Policy Self-Distillation. arXiv:2610.05070.

Yang, A.; Yang, B.; Zhang, B.; Hui, B.; Zheng, B.; Yu, B.; Li, C.; Liu, D.; Huang, F.; Dong, G.; et al. 2025. Qwen3 Technical Report. arXiv:2505.09388.

Zhao, S.; Xie, Z.; Liu, M.; Huang, J.; Pang, G.; Chen, F.; and Grover, A. 2026. Self-Distilled Reasoner: On-Policy Self-Distillation for Large Language Models. arXiv:2601.18734.